Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп’ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 9

Тема: Побудова надійного програмного забезпечення: Обробка виняткових ситуацій

Мета роботи: Опанувати практичні навички проектування стійких до відмов програмних комплексів мовою Python. Навчитися реалізовувати багаторівневу обробку помилок часу виконання за допомогою конструкції try-except-else-finally, проектувати ієрархію користувацьких класів винятків (Custom Exceptions) для конкретної інженерної предметної області, застосовувати механізм явної генерації винятків raise та зв'язування первинного контексту помилок через ланцюжки винятків (exception chaining). Опанувати парадигму EAFP (Easier to Ask for Forgiveness than Permission) для валідації параметрів апаратних пристроїв та конфігураційних файлів комп'ютерних систем.

Студента III курсу КІ-24-1 групи
Ступінь вищої освіти бакалавр
(бакалавр, магістр)
Спеціальність 123 – «Комп’ютерна інженерія»
Освітня програма Комп’ютерна інженерія
Яцентюка Євгенія Олександровича

м. Кременчук 2026 рік

1. Мета роботи та технічна постановка задачі

Мета роботи: опанувати проєктування стійких до відмов програмних комплексів мовою Python: реалізувати багаторівневу обробку помилок конструкцією try-except-else-finally, спроєктувати ієрархію користувацьких класів винятків для предметної області «швидкісний аналого-цифровий перетворювач», застосувати явну генерацію винятків raise та ланцюжки винятків (exception chaining), а також парадигму EAFP для валідації параметрів апаратного вузла.

1.1. Індивідуальне завдання (варіант 2)

Апаратний вузол	Контрольовані параметри та межі	Класи винятків
Модуль швидкісного АЦП (ADC)	Частота вибірки: 1 … 100 МГц; вхідна напруга: −2,5 … +2,5 В	ADCValidationError, SamplingRateError, InputOverloadError

У таблиці варіанта названо один базовий клас і два похідні, тоді як вимоги до реалізації передбачають щонайменше один базовий доменний клас і три похідні зі специфічними числовими полями стану. Тому до ієрархії додано третій похідний клас NyquistViolationError. Він випливає з фізики перетворювача: за критерієм Найквіста частота вхідного сигналу повинна бути меншою за половину частоти вибірки. Відповідна перевірка активується, якщо в конфігурації вказано необов’язковий параметр signal_freq_mhz.

1.2. Математичний апарат

Ймовірність безвідмовної роботи системи протягом часу t за експоненційним законом надійності та середній наробіток до відмови:

$$R(t) = e^{(-\lambda \cdot t)}, \quad MTBF = 1 / \lambda$$

Коефіцієнт програмного відновлення характеризує частку нештатних ситуацій, які перехоплено й оброблено блоками except за визначеною дією відновлення:

$$P_recovery = N_handled / N_total_faults$$

Фізичні інваріанти валідації АЦП для варіанта 2 та допоміжні розрахункові величини:

$$1\ МГц \le f_s \le 100\ МГц, \quad -2{,}5\ В \le U_in \le +2{,}5\ В, \quad f_signal < f_s / 2$$

$$T_s = 10^3 / f_s\ [нс], \quad f_Nyquist = f_s / 2$$

Для логічно зв’язаних винятків застосовується механізм ланцюжка (PEP 3134): оператор raise TargetException from OriginalException записує первинний виняток в атрибут __cause__ цільового винятку.

2. Діаграма класів винятків (UML)

Усі користувацькі винятки наслідуються від Exception, а не від BaseException. Тому вони не перехоплюють системних сигналів KeyboardInterrupt та SystemExit: оператор може перервати програму комбінацією Ctrl+C, а виклик sys.exit() завершить процес. Базовий доменний клас ADCValidationError інкапсулює спільні для всієї предметної області поля — числовий код помилки та час виникнення, — а три нащадки додають специфічні для свого збою числові поля стану.

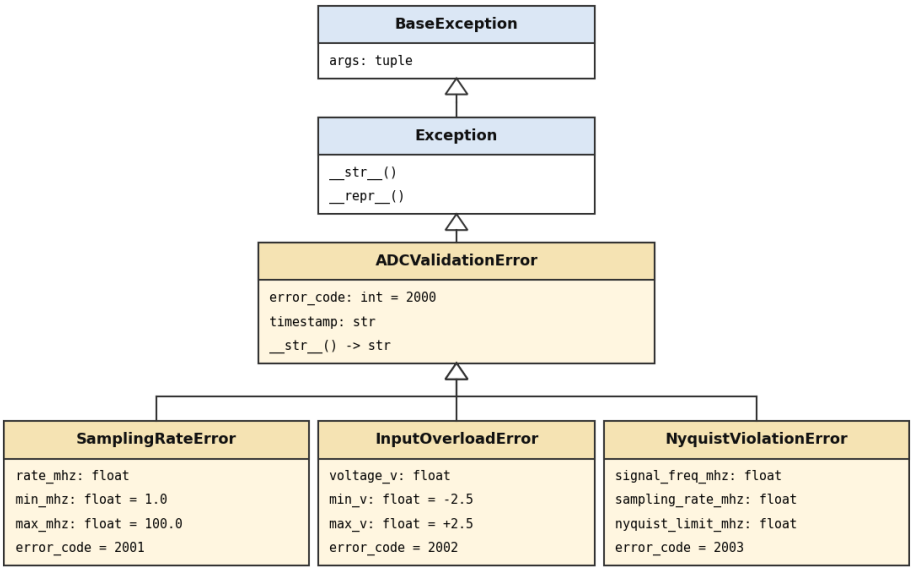


Рисунок 1 — Ієрархія користувацьких класів винятків для діагностики конфігурації АЦП

Клас	Код помилки	Специфічні атрибути та призначення
ADCValidationError	2000 (2004, 2005)	error_code, timestamp; базовий клас; напряму використовується для нечислових параметрів
SamplingRateError	2001	rate_mhz, min_mhz, max_mhz: частота вибірки поза діапазоном [1; 100] МГц
InputOverloadError	2002	voltage_v, min_v, max_v: вхідна напруга поза діапазоном [−2,5; +2,5] В
NyquistViolationError	2003	signal_freq_mhz, sampling_rate_mhz, nyquist_limit_mhz: порушення критерію Найквіста

Метод __str__ базового класу формує єдиний формат повідомлення «[ADC-ERR-код] (час) текст», тому нащадкам достатньо передати в конструктор базового класу готовий текст і код. Числові атрибути (наприклад, rate_mhz або nyquist_limit_mhz) використовуються обробником для обчислення дії відновлення, а не лише для виведення повідомлення.

3. Алгоритм валідації та обробки збоїв

3.1. Багаторівнева валідація за парадигмою EAFP

Функція validate_adc_configuration виконує перевірки послідовно: кожен наступний рівень припускає, що попередній пройдено.

Рівень	Перевірка	Виняток при порушенні
1	тип конфігурації (isinstance dict) та наявність обов’язкових ключів	TypeError; KeyError
2	перетворення параметрів у float (EAFP)	ADCValidationError (2004, 2005) from ValueError / TypeError
3	фізичні межі частоти вибірки та напруги	SamplingRateError (2001); InputOverloadError (2002) — обидва from ValueError
4	інваріант Найквіста f_signal < f_s / 2	NyquistViolationError (2003)

Парадигма EAFP («легше попросити пробачення, ніж дозвіл») означає, що замість попередньої перевірки придатності значення (регулярний вираз, isinstance, isdigit) програма просто виконує перетворення float(...) та обробляє виняток, якщо воно не вдалося. Це коротше й надійніше: перевірка за зразком не охопить усіх допустимих записів (наприклад, «1e-3» або «+5»). Окрім того, у середовищі з паралельним доступом EAFP стійкий до стану гонитви TOCTOU, бо перевірка та дія виконуються одним атомарним кроком, між якими не може змінитися стан.

Перевірка меж реалізована у два кроки відповідно до схеми з методичних вказівок: низькорівнева функція _ensure_within генерує стандартний ValueError з текстом про фактичне й допустиме значення, а верхній рівень перехоплює його та підіймає доменний виняток за допомогою raise SamplingRateError(...) from range_err. У результаті прикладна програма отримує інформативний виняток із числовими полями, а діагностична першопричина (ValueError) зберігається в атрибуті __cause__. Вираз low <= value <= high хибний для NaN, тому нечислові значення NaN теж відхиляються.

3.2. Конструкція try-except-else-finally у тестовому стенді

Для кожного набору параметрів тестовий стенд run_fault_tolerance_testbench виконує валідацію в блоці try. Порядок блоків except має принципове значення: інтерпретатор вибирає перший блок, клас якого збігається з класом винятку або є його предком. Тому послідовність така: SamplingRateError, InputOverloadError, NyquistViolationError (спеціалізовані нащадки), ADCValidationError (базовий доменний клас), TypeError, KeyError (стандартні) та останнім Exception. Якби блок except ADCValidationError стояв першим, спеціалізовані обробники ніколи б не виконувались, оскільки нащадок також є екземпляром базового класу.

Блок else виконується лише тоді, коли в try не виникло жодного винятку, тому звіт про успішну конфігурацію розміщено в ньому: помилка в коді звіту

не буде помилково перехоплена обробниками валідації. Блок finally виконується безумовно (після успіху, після обробленого винятку та після непередбаченого збою) і гарантує завершення аудиту й звільнення каналів АЦП.

3.3. Дії відновлення та метрика P_recovery

Виняток	**Дія відновлення (fail-safe)**
SamplingRateError	обмеження частоти до найближчої межі: max(min_mhz, min(rate_mhz, max_mhz))
InputOverloadError	увімкнення атенюатора, обмеження входу до найближчої межі ±2,5 В
NyquistViolationError	розрахунок мінімальної потрібної частоти 2·f_signal або антиаліасинговий фільтр
ADCValidationError (2004)	відхилення конфігурації та збереження попередніх налаштувань
TypeError, KeyError	відхилення пакета конфігурації; запит повної конфігурації від джерела
Exception (останній рубіж)	дія не визначена; збій вважається нелокалізованим

До N_handled належать лише збої, для яких визначено дію відновлення, а збої, спіймані заключним блоком except Exception, зараховуються до N_total_faults, але не до N_handled. Для демонстрації сенсу метрики в тестовий набір додано непередбачений апаратний збій: об’єкт датчика FlakySensor, перетворення якого на float() генерує TimeoutError (імітація таймауту шини SPI). Валідатор перехоплює лише ValueError та TypeError, тому такий збій потрапляє до останнього рубежу, і P_recovery стає меншим за 100 %. Це вказує розробнику на місце, де валідатор слід доопрацювати.

4. Програмний код

4.1. Файл custom_exceptions.py

```
"""
Лабораторна робота № 9. Модуль 1. Варіант 2.
Ієрархія користувацьких класів винятків для системи валідації конфігурації
```

```
модуля швидкісного аналого-цифрового перетворювача (ADC).

Фізичні обмеження варіанта 2:
    частота вибірки:   1 ... 100 МГц
    вхідна напруга:    -2.5 ... +2.5 В
Додатковий інваріант предметної області: за критерієм Найквіста частота сигналу
повинна бути строго меншою за половину частоти вибірки (f_signal < f_s / 2).

Освітній компонент: Програмування на Python.
"""

import time


class ADCValidationError(Exception):
    """
    Базовий доменний клас для всіх винятків валідації конфігурації АЦП.
    Наслідується від Exception (а не від BaseException), щоб блоки except не
    перехоплювали системні сигнали KeyboardInterrupt та SystemExit.
    """

    def __init__(self, message: str, error_code: int = 2000):
        super().__init__(message)
        self.error_code = error_code
        self.timestamp = time.strftime("%Y-%m-%d %H:%M:%S")

    def __str__(self) -> str:
        return f"[ADC-ERR-{self.error_code}] ({self.timestamp})
{super().__str__()}"


class SamplingRateError(ADCValidationError):
    """Збуджується, коли частота вибірки виходить за допустимий діапазон АЦП."""

    def __init__(self, rate_mhz: float, min_mhz: float = 1.0, max_mhz: float =
100.0):
        self.rate_mhz = rate_mhz
        self.min_mhz = min_mhz
        self.max_mhz = max_mhz
        msg = (f"Частота вибірки {rate_mhz:.2f} МГц виходить за межі допустимого
"
               f"діапазону [{min_mhz:.2f} .. {max_mhz:.2f}] МГц.")
        super().__init__(msg, error_code=2001)


class InputOverloadError(ADCValidationError):
    """Збуджується, коли вхідна напруга перевищує діапазон повної шкали АЦП."""

    def __init__(self, voltage_v: float, min_v: float = -2.5, max_v: float =
2.5):
        self.voltage_v = voltage_v
```

```
        self.min_v = min_v
        self.max_v = max_v
        msg = (f"Вхідна напруга {voltage_v:+.2f} В перевантажує вхід АЦП: "
               f"допустимий діапазон [{min_v:+.2f} .. {max_v:+.2f}] В.")
        super().__init__(msg, error_code=2002)


class NyquistViolationError(ADCValidationError):
    """Збуджується, коли частота сигналу не менша за межу Найквіста (f_s / 2)."""

    def __init__(self, signal_freq_mhz: float, sampling_rate_mhz: float):
        self.signal_freq_mhz = signal_freq_mhz
        self.sampling_rate_mhz = sampling_rate_mhz
        self.nyquist_limit_mhz = sampling_rate_mhz / 2.0
        msg = (f"Частота сигналу {signal_freq_mhz:.2f} МГц не менша за межу
Найквіста "
               f"{self.nyquist_limit_mhz:.2f} МГц (f_s = {sampling_rate_mhz:.2f}
МГц): "
               f"можливе перекриття спектра (aliasing).")
        super().__init__(msg, error_code=2003)
```

4.2. Файл device_validator.py

```
"""
Лабораторна робота № 9. Модуль 2. Варіант 2.
Багаторівнева валідація конфігурації модуля швидкісного АЦП (парадигма EAFP),
ланцюжки винятків (raise ... from ...) та тестовий стенд відмовостійкості
з розрахунком коефіцієнта програмного відновлення P_recovery.
Освітній компонент: Програмування на Python.
"""

import time
from typing import Any, Dict

from custom_exceptions import (
    ADCValidationError,
    SamplingRateError,
    InputOverloadError,
    NyquistViolationError,
)

# Фізичні межі варіанта 2
RATE_MIN_MHZ, RATE_MAX_MHZ = 1.0, 100.0
VOLT_MIN_V, VOLT_MAX_V = -2.5, 2.5
REQUIRED_KEYS = {"sampling_rate_mhz", "input_voltage_v"}


def _ensure_within(name: str, value: float, low: float, high: float) -> None:
    """
    Низькорівнева перевірка належності значення діапазону [low, high].
```

```
    Генерує стандартний ValueError, який верхні рівні перетворюють
    на доменний виняток із збереженням першопричини (raise ... from ...).
    Для NaN вираз low <= value <= high хибний, тому NaN також відхиляється.
    """
    if not (low <= value <= high):
        raise ValueError(f"{name} = {value} не належить діапазону [{low},
{high}]")


def validate_adc_configuration(config: Dict[str, Any]) -> Dict[str, float]:
    """
    Виконує послідовну валідацію конфігурації АЦП і повертає розраховані
характеристики:
    частоту вибірки, вхідну напругу, період вибірки (нс) та смугу Найквіста
(МГц).
    Рівні перевірки: структура -> типи -> фізичні межі -> інваріант Найквіста.
    """
    # Рівень 1. Структура: тип контейнера та наявність обов'язкових ключів
    if not isinstance(config, dict):
        raise TypeError(f"Конфігурація повинна бути словником, отримано
{type(config).__name__}.")

    missing_keys = REQUIRED_KEYS.difference(config.keys())
    if missing_keys:
        raise KeyError(f"У конфігурації відсутні обов'язкові параметри:
{sorted(missing_keys)}")

    # Рівень 2. Типи: EAFP - спершу пробуємо перетворити, помилку обробляємо
потім
    try:
        sampling_rate = float(config["sampling_rate_mhz"])
        input_voltage = float(config["input_voltage_v"])
    except (ValueError, TypeError) as conv_err:
        # Ланцюжок винятків: у __cause__ зберігається первинна помилка
перетворення
        raise ADCValidationError(
            "Параметри 'sampling_rate_mhz' та 'input_voltage_v' повинні бути
числовими.",
            error_code=2004,
        ) from conv_err

    # Рівень 3. Фізичні межі: низькорівневий ValueError перетворюється на
доменний виняток
    try:
        _ensure_within("sampling_rate_mhz", sampling_rate, RATE_MIN_MHZ,
RATE_MAX_MHZ)
    except ValueError as range_err:
        raise SamplingRateError(sampling_rate, RATE_MIN_MHZ, RATE_MAX_MHZ) from
range_err

    try:
```

```
        _ensure_within("input_voltage_v", input_voltage, VOLT_MIN_V, VOLT_MAX_V)
    except ValueError as range_err:
        raise InputOverloadError(input_voltage, VOLT_MIN_V, VOLT_MAX_V) from
range_err

    # Рівень 4. Інваріант Найквіста (необов'язковий параметр signal_freq_mhz)
    if "signal_freq_mhz" in config:
        try:
            signal_freq = float(config["signal_freq_mhz"])
        except (ValueError, TypeError) as conv_err:
            raise ADCValidationError(
                "Параметр 'signal_freq_mhz' повинен бути числовим.",
error_code=2005
            ) from conv_err
        if signal_freq >= sampling_rate / 2.0:
            raise NyquistViolationError(signal_freq, sampling_rate)

    return {
        "sampling_rate_mhz": sampling_rate,
        "input_voltage_v": input_voltage,
        "sample_period_ns": 1000.0 / sampling_rate,
        "nyquist_mhz": sampling_rate / 2.0,
    }


def describe_cause(err: BaseException) -> str:
    """Повертає рядок із першопричиною (__cause__) виняткової ситуації, якщо вона
є."""
    cause = err.__cause__
    return f"{type(cause).__name__}: {cause}" if cause is not None else ""


def run_fault_tolerance_testbench(test_configs: list) -> None:
    """
    Запускає тестовий стенд: кожен набір проходить через try-except-else-finally.
    Підраховує успішні конфігурації, локалізовані збої та коефіцієнт відновлення
    P_recovery = N_handled / N_total_faults.
    """
    header_sep = "=" * 90
    sub_sep = "-" * 90

    print(header_sep)
    print(f"{'ТЕСТУВАННЯ ВІДМОВОСТІЙКОСТІ ТА ОБРОБКИ ВИНЯТКІВ МОДУЛЯ АЦП':^90}")
    print(header_sep)

    successful_runs = 0
    handled_faults = 0
    unclassified_faults = 0
    fault_stats: Dict[str, int] = {}
    total_configs = len(test_configs)
```

```
    for idx, cfg in enumerate(test_configs, start=1):
        print(f"\nТест #{idx:02d}: конфігурація -> {cfg}")
        try:
            result = validate_adc_configuration(cfg)

        # Порядок except: від найспеціалізованіших нащадків до найзагальніших
класів
        except SamplingRateError as err:
            handled_faults += 1
            fault_stats["SamplingRateError"] =
fault_stats.get("SamplingRateError", 0) + 1
            print(f"  [ПЕРЕХОПЛЕНО СПЕЦІАЛІЗОВАНО]: {err}")
            safe_rate = max(err.min_mhz, min(err.rate_mhz, err.max_mhz))
            print(f"  -> Дія системи: обмеження частоти вибірки до
{safe_rate:.2f} МГц.")
            print(f"  -> Першопричина: {describe_cause(err)}")
        except InputOverloadError as err:
            handled_faults += 1
            fault_stats["InputOverloadError"] =
fault_stats.get("InputOverloadError", 0) + 1
            print(f"  [ПЕРЕХОПЛЕНО СПЕЦІАЛІЗОВАНО]: {err}")
            safe_v = max(err.min_v, min(err.voltage_v, err.max_v))
            print(f"  -> Дія системи: увімкнення атенюатора, обмеження входу до
{safe_v:+.2f} В.")
            print(f"  -> Першопричина: {describe_cause(err)}")
        except NyquistViolationError as err:
            handled_faults += 1
            fault_stats["NyquistViolationError"] =
fault_stats.get("NyquistViolationError", 0) + 1
            print(f"  [ПЕРЕХОПЛЕНО СПЕЦІАЛІЗОВАНО]: {err}")
            need_rate = 2.0 * err.signal_freq_mhz
            print(f"  -> Дія системи: потрібна частота вибірки > {need_rate:.2f}
МГц "
                  f"або антиаліасинговий фільтр.")
        except ADCValidationError as err:
            handled_faults += 1
            fault_stats["ADCValidationError"] =
fault_stats.get("ADCValidationError", 0) + 1
            print(f"  [ПЕРЕХОПЛЕНО БАЗОВИЙ ДОМЕННИЙ]: {err}")
            print(f"  -> Першопричина (Chained Cause): {describe_cause(err)}")
            print("  -> Дія системи: відхилення конфігурації, збереження
попередніх налаштувань.")
        except TypeError as err:
            handled_faults += 1
            fault_stats["TypeError"] = fault_stats.get("TypeError", 0) + 1
            print(f"  [ПЕРЕХОПЛЕНО СТАНДАРТНИЙ TypeError]: {err}")
            print("  -> Дія системи: відхилення пакета конфігурації.")
        except KeyError as err:
            handled_faults += 1
            fault_stats["KeyError"] = fault_stats.get("KeyError", 0) + 1
            print(f"  [ПЕРЕХОПЛЕНО СТАНДАРТНИЙ KeyError]: {err}")
```

```
            print("  -> Дія системи: запит повної конфігурації від джерела.")
        except Exception as err:
            # Останній рубіж: непередбачена помилка, для якої не визначено дії
відновлення
            unclassified_faults += 1
            print(f"  [НЕКЛАСИФІКОВАНИЙ ЗБІЙ]: {type(err).__name__}: {err}")
            print("  -> Дія системи: не визначена (потребує доопрацювання
валідатора).")
        else:
            successful_runs += 1
            print(f"  [УСПІХ]: Конфігурація валідна. f_s =
{result['sampling_rate_mhz']:.2f} МГц, "
                  f"період {result['sample_period_ns']:.2f} нс, "
                  f"смуга Найквіста {result['nyquist_mhz']:.2f} МГц.")
        finally:
            print(f"  [FINALLY]: Аудит тесту #{idx} завершено. Канали АЦП
звільнено.")

    total_faults = handled_faults + unclassified_faults
    p_recovery = handled_faults / total_faults if total_faults > 0 else 1.0

    print("\n" + header_sep)
    print(f"{'ПІДСУМКОВІ МЕТРИКИ НАДІЙНОСТІ СИСТЕМИ':^90}")
    print(sub_sep)
    print(f"  Всього оброблено тестових наборів:         {total_configs:>5d}")
    print(f"  Штатних успішних конфігурацій:             {successful_runs:>5d}")
    print(f"  Усього нештатних ситуацій (N_total):       {total_faults:>5d}")
    print(f"  Локалізованих та оброблених (N_handled):   {handled_faults:>5d}")
    print(f"  Некласифікованих збоїв:
 {unclassified_faults:>5d}")
    for name, count in sorted(fault_stats.items()):
        print(f"      {name:<28} {count:>5d}")
    print(f"  Коефіцієнт програмного відновлення (P):   {p_recovery * 100:>7.2f}
%")
    print(header_sep + "\n")


class FlakySensor:
    """
    Імітація апаратного датчика з несправною шиною: перетворення float() генерує
    TimeoutError, який валідатор не передбачає (не ValueError і не TypeError).
    """

    def __float__(self) -> float:
        raise TimeoutError("таймаут шини SPI під час зчитування датчика напруги")

    def __repr__(self) -> str:
        return "<FlakySensor: шина не відповідає>"


def main() -> None:
```

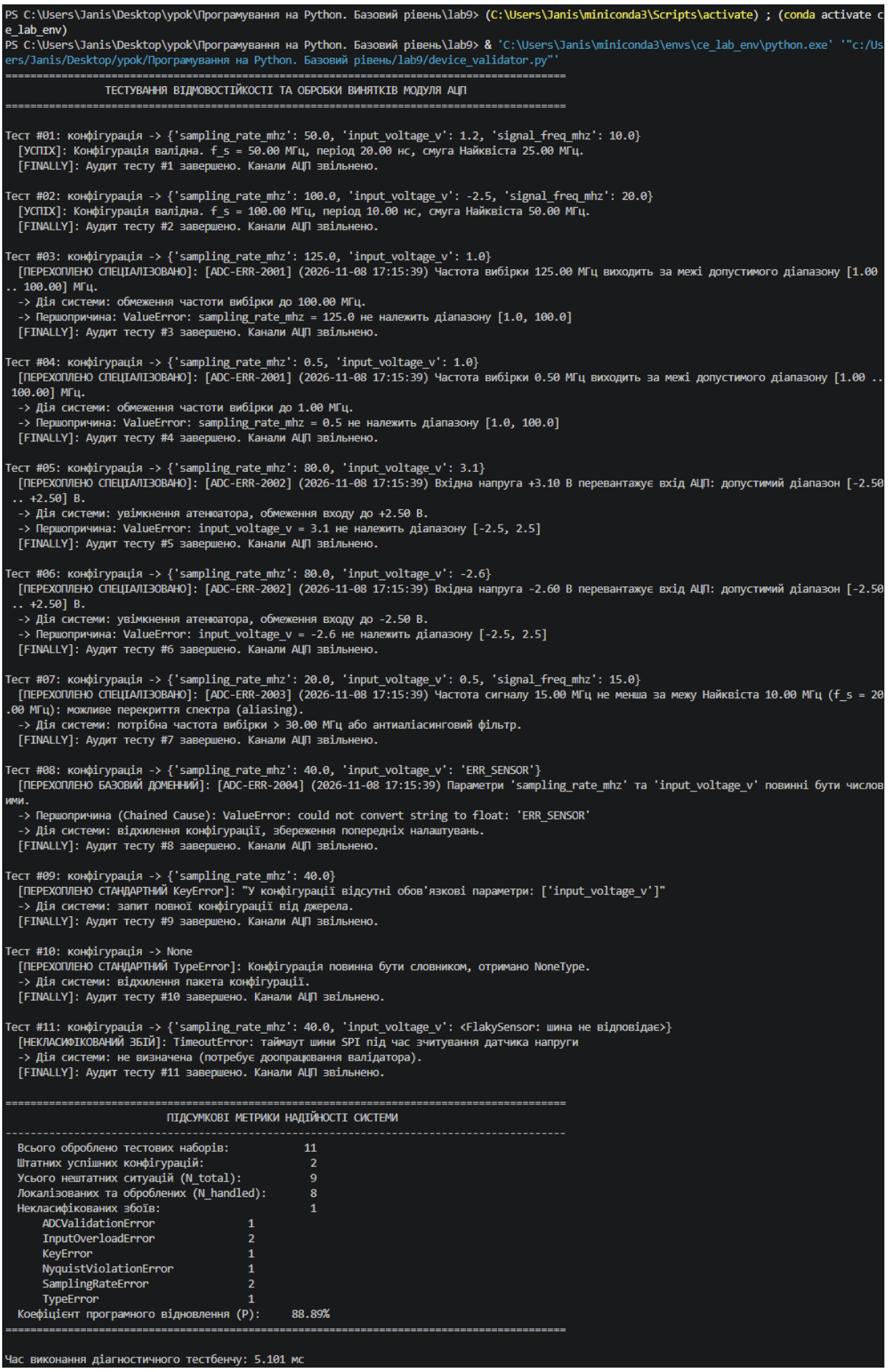
```
    """Головна точка входу лабораторної роботи."""
    test_suite = [
        # 1: коректна конфігурація
        {"sampling_rate_mhz": 50.0, "input_voltage_v": 1.2, "signal_freq_mhz":
10.0},
        # 2: коректна конфігурація на граничних значеннях (межі включено)
        {"sampling_rate_mhz": 100.0, "input_voltage_v": -2.5, "signal_freq_mhz":
20.0},
        # 3: частота вибірки вище максимуму
        {"sampling_rate_mhz": 125.0, "input_voltage_v": 1.0},
        # 4: частота вибірки нижче мінімуму
        {"sampling_rate_mhz": 0.5, "input_voltage_v": 1.0},
        # 5: перевантаження входу позитивною напругою
        {"sampling_rate_mhz": 80.0, "input_voltage_v": 3.1},
        # 6: перевантаження входу негативною напругою
        {"sampling_rate_mhz": 80.0, "input_voltage_v": -2.6},
        # 7: порушення критерію Найквіста (15 МГц >= 20 / 2)
        {"sampling_rate_mhz": 20.0, "input_voltage_v": 0.5, "signal_freq_mhz":
15.0},
        # 8: нечислове значення напруги (ланцюжок із ValueError)
        {"sampling_rate_mhz": 40.0, "input_voltage_v": "ERR_SENSOR"},
        # 9: відсутній обов'язковий ключ
        {"sampling_rate_mhz": 40.0},
        # 10: некоректний тип самої конфігурації
        None,
        # 11: непередбачений апаратний збій: датчик не відповідає (TimeoutError)
        {"sampling_rate_mhz": 40.0, "input_voltage_v": FlakySensor()},
    ]

    t_start = time.perf_counter()
    run_fault_tolerance_testbench(test_suite)
    t_elapsed = time.perf_counter() - t_start
    print(f"Час виконання діагностичного тестбенчу: {t_elapsed * 1000:.3f} мс\n")


if __name__ == "__main__":
    main()
```

5. Результати тестування

Запуск виконувався в середовищі Conda ce_lab_env командою python device_validator.py. Тестовий набір містить 11 конфігурацій: дві коректні, вісім із передбаченими дефектами та одну з непередбаченим апаратним збоєм.



```
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab9> (C:\Users\Janis\miniconda3\Scripts\activate) ; (conda activate c
e_lab_env)
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab9> & 'C:\Users\Janis\miniconda3\envs\ce_lab_env\python.exe' '"c:/Us
ers/Janis/Desktop/урок/Програмування на Python. Базовий рівень/lab9/device_validator.py"'
================================================================================
               ТЕСТУВАННЯ ВІДМОВОСТІЙКОСТІ ТА ОБРОБКИ ВИНЯТКІВ МОДУЛЯ АЦП
================================================================================

Тест #01: конфігурація -> {'sampling_rate_mhz': 50.0, 'input_voltage_v': 1.2, 'signal_freq_mhz': 10.0}
  [УСПІХ]: Конфігурація валідна. f_s = 50.00 МГц, період 20.00 нс, смуга Найквіста 25.00 МГц.
  [FINALLY]: Аудит тесту #1 завершено. Канали АЦП звільнено.

Тест #02: конфігурація -> {'sampling_rate_mhz': 100.0, 'input_voltage_v': -2.5, 'signal_freq_mhz': 20.0}
  [УСПІХ]: Конфігурація валідна. f_s = 100.00 МГц, період 10.00 нс, смуга Найквіста 50.00 МГц.
  [FINALLY]: Аудит тесту #2 завершено. Канали АЦП звільнено.

Тест #03: конфігурація -> {'sampling_rate_mhz': 125.0, 'input_voltage_v': 1.0}
  [ПЕРЕХОПЛЕНО СПЕЦІАЛІЗОВАНО]: [ADC-ERR-2001] (2026-11-08 17:15:39) Частота вибірки 125.00 МГц виходить за межі допустимого діапазону [1.00
.. 100.00] МГц.
  -> Дія системи: обмеження частоти вибірки до 100.00 МГц.
  -> Першопричина: ValueError: sampling_rate_mhz = 125.0 не належить діапазону [1.0, 100.0]
  [FINALLY]: Аудит тесту #3 завершено. Канали АЦП звільнено.

Тест #04: конфігурація -> {'sampling_rate_mhz': 0.5, 'input_voltage_v': 1.0}
  [ПЕРЕХОПЛЕНО СПЕЦІАЛІЗОВАНО]: [ADC-ERR-2001] (2026-11-08 17:15:39) Частота вибірки 0.50 МГц виходить за межі допустимого діапазону [1.00 ..
 100.00] МГц.
  -> Дія системи: обмеження частоти вибірки до 1.00 МГц.
  -> Першопричина: ValueError: sampling_rate_mhz = 0.5 не належить діапазону [1.0, 100.0]
  [FINALLY]: Аудит тесту #4 завершено. Канали АЦП звільнено.

Тест #05: конфігурація -> {'sampling_rate_mhz': 80.0, 'input_voltage_v': 3.1}
  [ПЕРЕХОПЛЕНО СПЕЦІАЛІЗОВАНО]: [ADC-ERR-2002] (2026-11-08 17:15:39) Вхідна напруга +3.10 В перевантажує вхід АЦП: допустимий діапазон [-2.50
 .. +2.50] В.
  -> Дія системи: увімкнення атенюатора, обмеження входу до +2.50 В.
  -> Першопричина: ValueError: input_voltage_v = 3.1 не належить діапазону [-2.5, 2.5]
  [FINALLY]: Аудит тесту #5 завершено. Канали АЦП звільнено.

Тест #06: конфігурація -> {'sampling_rate_mhz': 80.0, 'input_voltage_v': -2.6}
  [ПЕРЕХОПЛЕНО СПЕЦІАЛІЗОВАНО]: [ADC-ERR-2002] (2026-11-08 17:15:39) Вхідна напруга -2.60 В перевантажує вхід АЦП: допустимий діапазон [-2.50
 .. +2.50] В.
  -> Дія системи: увімкнення атенюатора, обмеження входу до -2.50 В.
  -> Першопричина: ValueError: input_voltage_v = -2.6 не належить діапазону [-2.5, 2.5]
  [FINALLY]: Аудит тесту #6 завершено. Канали АЦП звільнено.

Тест #07: конфігурація -> {'sampling_rate_mhz': 20.0, 'input_voltage_v': 0.5, 'signal_freq_mhz': 15.0}
  [ПЕРЕХОПЛЕНО СПЕЦІАЛІЗОВАНО]: [ADC-ERR-2003] (2026-11-08 17:15:39) Частота сигналу 15.00 МГц не менша за межу Найквіста 10.00 МГц (f_s = 20
.00 МГц): можливе перекриття спектра (aliasing).
  -> Дія системи: потрібна частота вибірки > 30.00 МГц або антиаліасинговий фільтр.
  [FINALLY]: Аудит тесту #7 завершено. Канали АЦП звільнено.

Тест #08: конфігурація -> {'sampling_rate_mhz': 40.0, 'input_voltage_v': 'ERR_SENSOR'}
  [ПЕРЕХОПЛЕНО БАЗОВИЙ ДОМЕННИЙ]: [ADC-ERR-2004] (2026-11-08 17:15:39) Параметри 'sampling_rate_mhz' та 'input_voltage_v' повинні бути числов
ими.
  -> Першопричина (Chained Cause): ValueError: could not convert string to float: 'ERR_SENSOR'
  -> Дія системи: відхилення конфігурації, збереження попередніх налаштувань.
  [FINALLY]: Аудит тесту #8 завершено. Канали АЦП звільнено.

Тест #09: конфігурація -> {'sampling_rate_mhz': 40.0}
  [ПЕРЕХОПЛЕНО СТАНДАРТНИЙ KeyError]: "У конфігурації відсутні обов'язкові параметри: ['input_voltage_v']"
  -> Дія системи: запит повної конфігурації від джерела.
  [FINALLY]: Аудит тесту #9 завершено. Канали АЦП звільнено.

Тест #10: конфігурація -> None
  [ПЕРЕХОПЛЕНО СТАНДАРТНИЙ TypeError]: Конфігурація повинна бути словником, отримано NoneType.
  -> Дія системи: відхилення пакета конфігурації.
  [FINALLY]: Аудит тесту #10 завершено. Канали АЦП звільнено.

Тест #11: конфігурація -> {'sampling_rate_mhz': 40.0, 'input_voltage_v': <FlakySensor: шина не відповідає>}
  [НЕКЛАСИФІКОВАНИЙ ЗБІЙ]: TimeoutError: таймаут шини SPI під час зчитування датчика напруги
  -> Дія системи: не визначена (потребує доопрацювання валідатора).
  [FINALLY]: Аудит тесту #11 завершено. Канали АЦП звільнено.

================================================================================
                        ПІДСУМКОВІ МЕТРИКИ НАДІЙНОСТІ СИСТЕМИ
--------------------------------------------------------------------------------
  Всього оброблено тестових наборів:           11
  Штатних успішних конфігурацій:                2
  Усього нештатних ситуацій (N_total):          9
  Локалізованих та оброблених (N_handled):      8
  Некласифікованих збоїв:                       1
      ADCValidationError              1
      InputOverloadError              2
      KeyError                        1
      NyquistViolationError           1
      SamplingRateError               2
      TypeError                       1
  Коефіцієнт програмного відновлення (P):     88.89%
================================================================================

Час виконання діагностичного тестбенчу: 5.101 мс
```

Скріншот 2 — Результат роботи тестового стенда відмовостійкості модуля АЦП

Відповідність тестів та винятків:

Тест	Вхідні дані	Результат	Код
1, 2	f_s = 50 та 100 МГц; U = 1,2 та −2,5 В	УСПІХ (межі включено)	—
3, 4	f_s = 125 МГц; f_s = 0,5 МГц	SamplingRateError	ADC-ERR-2001
5, 6	U = +3,1 В; U = −2,6 В	InputOverloadError	ADC-ERR-2002
7	f_s = 20 МГц, f_signal = 15 МГц	NyquistViolationError	ADC-ERR-2003
8	U = "ERR_SENSOR"	ADCValidationError from ValueError	ADC-ERR-2004
9	немає input_voltage_v	KeyError	—
10	None замість словника	TypeError	—
11	датчик напруги не відповідає	TimeoutError (некласифікований)	—

Аналіз результатів. У тестах 1 і 2 конфігурація валідна: для f_s = 50 МГц період вибірки T_s = 1000 / 50 = 20 нс, а межа Найквіста 25 МГц; для граничного випадку f_s = 100 МГц період становить 10 нс. Обидва тести підтверджують, що крайні значення діапазонів (100 МГц і −2,5 В) допускаються. У тестах 3 і 4 дія відновлення обмежує частоту до найближчої межі (100 та 1 МГц), а в тестах 5 і 6 напругу обмежено до +2,50 та −2,50 В: знак відновленого значення збігається зі знаком вхідного. У тесті 7 межа Найквіста для 20 МГц дорівнює 10 МГц, тому сигнал 15 МГц неприпустимий, і потрібна частота вибірки перевищує 30 МГц.

У тесті 8 продемонстровано ланцюжок винятків: перехоплений ADCValidationError (код 2004) має в атрибуті __cause__ первинний ValueError із текстом про неможливість перетворення рядка в число; у тестах 3–6 причиною виступає низькорівневий ValueError із фактичним значенням і діапазоном. Блок finally спрацював в усіх 11 тестах.

Підсумкові метрики: з 11 наборів штатно пройшли 2, нештатних ситуацій виникло N_total = 9, оброблено N_handled = 8, один збій (тест 11) не має дії відновлення. Тому P_recovery = 8 / 9 = 88,89 %. Розподіл оброблених

збоїв: SamplingRateError — 2, InputOverloadError — 2, NyquistViolationError, ADCValidationError, TypeError та KeyError — по 1.

6. Посилання на віддалений репозиторій

Профіль: https://github.com/kefir4ikk

Репозиторій лабораторної роботи: https://github.com/kefir4ikk/progPython_YevgeniiYatsentiuk/tree/master/labs/lab9

7. Висновки

У ході лабораторної роботи розроблено систему надійної валідації конфігурації модуля швидкісного АЦП для варіанта 2. Спроєктована ієрархія з базового доменного класу ADCValidationError та трьох нащадків (SamplingRateError, InputOverloadError, NyquistViolationError) дозволяє не лише повідомити про помилку, а й передати обробнику структуровані числові дані: виміряне значення, фізичні межі, межу Найквіста. На їх основі система обчислює конкретну дію відновлення, чого неможливо досягти з повертанням числових кодів помилок або загальним текстовим повідомленням.

Користувацькі винятки мають переваги над кодами повернення: їх неможливо випадково проігнорувати (необроблений виняток перериває виконання, тоді як код повернення можна не перевірити), вони відокремлюють шлях помилки від основної логіки, а їхня ієрархія дозволяє перехоплювати як конкретний збій, так і цілий клас збоїв одним блоком except. Ланцюжки raise ... from ... зберігають першопричину (ValueError) на всіх шарах системи, що спрощує діагностику.

Підхід EAFP зробив валідацію типів лаконічною та стійкою: перетворення float() із обробкою винятку охоплює всі допустимі записи чисел і не залежить від проміжного стану. Конструкція try-except-else-finally забезпечила детермінований контроль: порядок блоків except від спеціалізованих до загальних, success-логіка в else і безумовне звільнення

ресурсів у finally. Розрахунок P_recovery = 8/9 = 88,89 % показав, що метрика чутлива до непередбачених збоїв: заключний блок except Exception не дає програмі аварійно завершитись, але чесно фіксує, що для таких випадків дія відновлення відсутня, і вказує, де валідатор слід розширити (наприклад, обробкою TimeoutError).

Контрольні питання

1. У чому полягає принципова різниця між системними винятками, похідними від BaseException (наприклад, KeyboardInterrupt, SystemExit), та прикладними винятками, похідними від Exception?

BaseException є коренем усієї ієрархії. Від нього безпосередньо успадковані Exception, а також системні винятки SystemExit (виклик sys.exit()), KeyboardInterrupt (Ctrl+C), GeneratorExit та, починаючи з Python 3.11, BaseExceptionGroup. Прикладні помилки програми (ValueError, KeyError, OSError тощо) наслідуються від Exception. Конструкція except Exception не перехоплює системні винятки, тому користувач може перервати програму, а sys.exit() завершить процес. Саме тому користувацькі винятки наслідують від Exception; блок except без вказаного класу або except BaseException перехопив би і сигнали завершення, що вважається небезпечною практикою.

2. Опишіть внутрішній механізм розкручування стека викликів (stack unwinding) у CPython. Що відбувається з об'єктами локального стекового кадру PyFrameObject при виникненні винятку?

При виконанні raise інтерпретатор створює об’єкт винятку й прикріплює до нього трасування (traceback), що складається з посилань на кадри, через які проходить виняток. Спочатку шукається обробник у поточному кадрі (у CPython 3.11+ за таблицею винятків скомпільованого байт-коду, раніше — за стеком блоків). Якщо обробника немає, виконуються блоки finally цього кадру, функція завершується аварійно, а виняток передається кадру викликача, де пошук повторюється. Якщо виняток досяг вершини стека, виводиться

трасування й процес завершується з ненульовим кодом. Кадри, через які пройшов виняток, не знищуються одразу: трасування утримує посилання на них, а отже й на їхні локальні змінні, доки живий об'єкт винятку. Тому ім'я в конструкції except ... as err автоматично видаляється наприкінці блоку, щоб не створювати циклічних посилань.

3. Поясніть призначення та синтаксис ланцюжків винятків (exception chaining, PEP 3134) за допомогою конструкції raise ... from .... Яка інформація зберігається в атрибуті __cause__?

Конструкція raise NewError(...) from original_error явно пов'язує новий виняток із первинним: у атрибут NewError.__cause__ записується об'єкт original_error разом із його трасуванням, а в __suppress_context__ встановлюється True. У трасуванні з'являється повідомлення «The above exception was the direct cause of the following exception». Без from при створенні винятку всередині блоку except інтерпретатор неявно заповнює __context__ («During handling of the above exception, another exception occurred»). Запис from None пригнічує контекст. У лабораторній роботі __cause__ зберігає ValueError, що спричинив SamplingRateError, InputOverloadError або ADCValidationError, тому прикладний код може вивести першопричину збою.

4. Порівняйте парадигми програмування EAFP (Easier to Ask for Forgiveness than Permission) та LBYL (Look Before You Leap). Чому в багатопотокових системах підхід EAFP є більш стійким до стану гонитви TOCTOU?

LBYL («дивись, перш ніж стрибнути») спочатку перевіряє умови (if key in d, if os.path.exists(path)), а потім виконує дію. EAFP виконує дію й обробляє виняток, якщо вона не вдалася. У багатопотокових або багатопроцесних системах між перевіркою та дією стан може змінитися (файл видалено, ключ вилучено іншим потоком): це гонитва TOCTOU (time-of-check to time-of-use). EAFP не має такого проміжку, бо перевірка й використання відбуваються в одній операції, а її результат повідомляє виняток. Крім того, EAFP швидший у типовому випадку, коли винятки рідкісні. Недолік: часті винятки дорожчі за

перевірку, а побічні ефекти часткового виконання потребують окремого контролю.

5. Чому блоки except повинні розташовуватися строго у порядку від найбільш спеціалізованих (похідних) класів до найбільш загальних (базових)? Що станеться, якщо першим блоком вказати except Exception:?

Інтерпретатор перевіряє блоки по черзі й виконує перший, клас якого збігається з класом винятку або є його предком (за isinstance). Нащадок завжди є екземпляром предка, тому якщо блок із загальним класом стоїть вище, блок зі спеціалізованим класом нижче стає недосяжним: його код ніколи не виконається, і не спрацює спеціалізована дія відновлення. Python не повідомляє про це помилкою під час виконання, але лінтери попереджають. Якщо першим вказати except Exception, він перехопить усі прикладні винятки: у цій лабораторній роботі жоден спеціалізований обробник не спрацював би, а метрика P_recovery втратила б сенс. Загальний клас розміщують останнім як страхувальний рубіж.

6. За яких умов виконується блок else у конструкції try-except-else-finally? Чому відокремлення коду успішного виконання в блок else є кращою практикою, ніж розміщення його в кінці блоку try?

Блок else виконується, якщо тіло try завершилося без винятку (і не було передчасно перервано операторами return, break чи continue). Він виконується до finally. Розміщення коду успіху в else — краща практика, бо в try залишаються лише ті інструкції, які можуть згенерувати очікувані винятки. Виняток, що виникне в коді else, не буде помилково перехоплений блоками except цього ж try: це зменшує ризик «замаскувати» чужу помилку обробником, призначеним для іншої. У лабораторній роботі виведення звіту про успішну конфігурацію винесено в else, тому помилка у форматуванні звіту не буде сприйнята як збій валідації.